

Correct Optimizations: Phase 2 Report Based on “Minotaur: A SIMD-Oriented Synthesizing Superoptimizer” [1]

Kilian Schulz
Technical University Munich

Ilja Gamza
Technical University Munich

Tomás Gaspar
Technical University Munich

1 Introduction

Minotaur [1] is a superoptimizer, that after computing optimizations, caches those in a data base. These optimizations consist of a key, which is the original code, and a value, which is an equivalent, but faster, rewrite of the original code. This caching is done in order to reduce the substantial runtimes of Minotaur for future compilations of the same or a similar program. Minotaurs compile times, with a cold cache, can easily be hundreds of time slower than with regular LLVM [1].

Currently Minotaur directly uses the extracted code segments in LLVM IR as the keys and stores those along side a rewrite, in Minotaurs own IR as value, in the cache as shown in example 1.

This approach can easily lead to near misses, where an equivalent code segment is already cached, but for instance has unused arguments, leading to a cache miss and a costly recompute of an optimization.

One way to avoid this issue is to attempt to increase the number of cache hits while keeping the correctness of the mapping between keys and their rewrite. This leads to reduced compile times of Minotaur even when starting from a cold cache.

2 Research Proposal

Our solution introduces an additional step after the cut extraction. It performs multiple code transformations, with the goal of reducing the number of possible equivalent keys, mapping those to one canonicalized key.

The idea is that this leads to fewer cache entries, increasing the cache hit rate and reducing the number of slow optimization searches by Minotaur. For large enough projects this can already take effect during a build even when starting with an empty cache thereby reducing compile time.

3 Canonicalization

Our extended version of Minotaur is available on GitHub ¹.

The canonicalizer is placed as a pre- and postprocessing step around Minotaurs inference step. We intercept the LLVM IR produced by the slicer and perform transformations on it to produce a canonicalized version of the input which is then forwarded to the inference step. In the following this transformation is called canonicalization. After the inference step is finishes and returns a rewrite (in Minotaurs own IR), the canonicalizer again performs transformations on the rewrites. This process is called de-canonicalization. The structure of the augmented Minotaur with our canonicalizer is shown in figure 2.

Each canonicalization-step in our implementation consists of a canonicalization function and a de-canonicalization function. The canonicalization-steps are ordered as to not interfere with each other. More on that is discussed in section 3.2. During the canonicalization process every canonicalization function is applied iteratively in this predefined order. For de-canonicalization the de-canonicalization functions are applied in the reverse order in which the canonicalization-steps are specified. It is also possible to pass information between the pairs of canonicalization and de-canonicalization functions of each step if required. This enables the support for more complex transformations.

3.1 Canonicalization Steps

In the following we explain each canonicalization-step. We selected our steps such that each step is correct, productive and does not modify the functions depth. Correctness means that both the canonical function and the original function applied to the same function arguments compute the same result(and have the same effect on memory also with respect to order of operation). A step is productive iff it reduces size of the space of all possible keys. The depth is kept constant

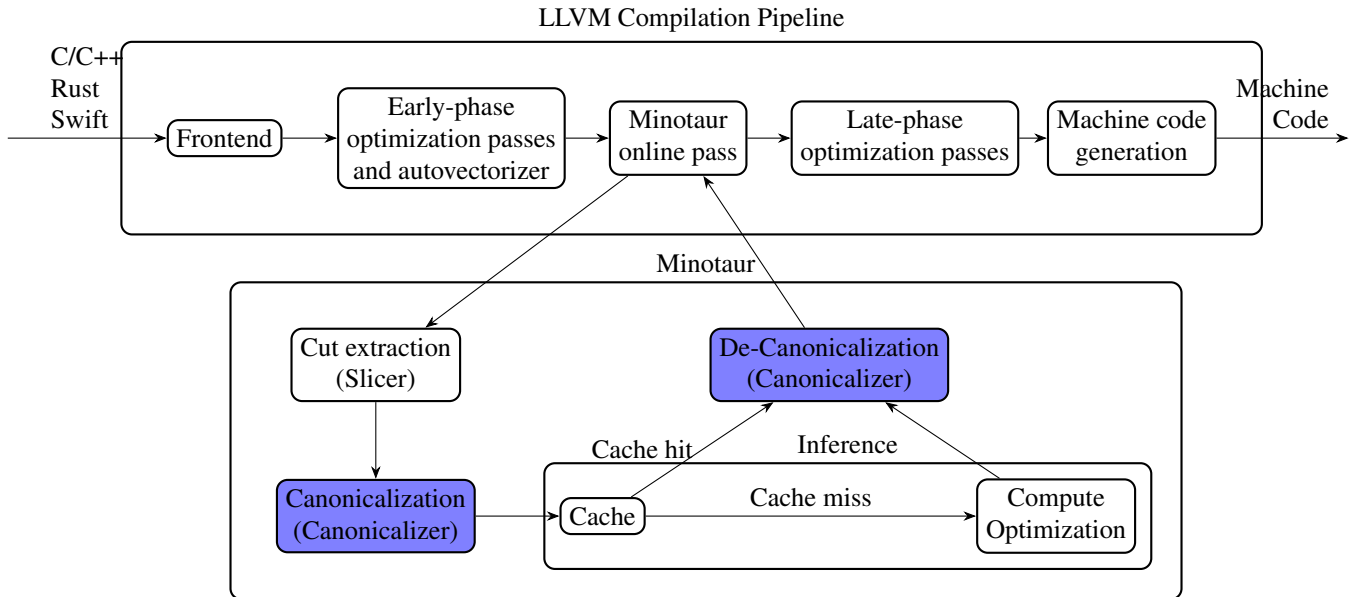
¹Extended Minotaur Version

Figure 1: Example of a key value pair as it is found in the Minotaur cache.

```
key:
    define i1 @cut(i64 %__n2, i16 %__n3) {
        %__n0 = sub nsw i64 0, %__n2
        %__n1 = icmp samesign ult i64 %__n0, 2033
        ret i1 %__n1
    }

value:
    (icmp_sgt (var i64 %__n2) (reservedconst i64 |i64 -2033|) b1)
```

Figure 2: Structure of Minotaur and how the Canonicalization is integrated. The highlighted blocks were added for (De-)Canonicalization.



as increasing this has a large impact on runtime as shown in original paper [1].

Those criteria were chosen to only allow for simple transformation steps. It might be possible to relax the correctness definition. Similarly the definition of productivity could be relaxed such that it only holds over all steps but not for every individual step. Also the depth constraint is not strictly required and could be dropped, but this would then likely result in larger solver queries which could drastically impact the overall build times.

An example of all the steps applied sequentially to an example input is shown in figure 3

Replacing Greater-Than with Less-Than A single comparison operation often has multiple semantically equivalent ways of being expressed. To address this we replace every greater-than or greater-equal comparison with a less-than or

less-equal comparison respectively. For correctness we also have to swap the order of operands.

$$\forall x \forall y. (x P y \iff y P' x) \\ \text{with } (P, P') \in (>, <), (\geq, \leq)$$

This reduces the size of the key space since now only less-than and less-equal comparisons are performed. It also does not change the depth of the function.

Non-strict Operator Replacement The idea is to reduce the number of comparison operators that are available in order to decrease the number of possible way to express a comparison operation. This is done by replacing non-strict comparison operators like $\leq$ with $<$. To introduce additional operations this is only done when a constant is one of the operands. This constant is then adjusted accordingly if no prevented by an

edge case like the constant being already the maximum or minimum value possible for the data type. So given some expression like $x \leq c$ it would be transformed into $x < c + \epsilon$ where ϵ is the smallest possible increment possible for c .

$$\exists v \exists \epsilon \forall x. ((v = c + \epsilon) \implies (x \leq c \iff x < c + \epsilon))$$

An analogous argument can be made for the $\geq$ and for a swapped operands and works for both integral and floating point types. Currently vector types are filtered.

Elimination of Unused Function Arguments Some function have unused arguments. Those can simply be eliminated during the forward pass. Assuming the given input function f has an unused argument we can define the following. Let $f : T_1 \times \dots \times T_i \times \dots \times T_n \rightarrow R$ and $g : T_1 \times \dots \times T_{i-1} \times T_{i+1} \times \dots \times T_n \rightarrow R$ be n -ary and $n - 1$ -ary functions respectively with

$$f(t_1, \dots, t_n) := g(t_1, \dots, t_{i-1}, t_{i+1}, \dots, t_n), t_j \in T_j$$

Since t_i is not part of the function signature of g we can define a new function $f' := g$. This obviously does not change what is computed since

$$\begin{aligned} f(t_1, \dots, t_n) &= g(t_1, \dots, t_{i-1}, t_{i+1}, \dots, t_n) \\ &= f'(t_1, \dots, t_{i-1}, t_{i+1}, \dots, t_n) \end{aligned}$$

Due to the implicit nature of the function signature in the values of the cache and the structure of Minotaur no additional work is required in relation to this step.

Sorting of Function Arguments Another modification that is done to the function signature is that we sort the arguments. The arguments are sorted by type-name and internally by when they are used inside the function body. Sorting by the type name is done in order to avoid cases where the function body is the same but a cache miss occurs due to the order of the types being different. To achieve a total order on the function arguments, each set of arguments of the same type are sorted by when they are used in the function body. This makes the argument order dependent on the function body and has to be taken into account when ordering the canonicalization steps. The correctness of the step is shown in the following. Let $f : T_1 \times \dots \times T_n \rightarrow R$ be the input function and $f' : T_{p(1)} \times \dots \times T_{p(n)} \rightarrow R$ the output function with $p : [n] \rightarrow [n]$ being a bijective function i.e. a permutation. We define

$$\begin{aligned} f'(t_{p(1)}, \dots, t_{p(n)}) &:= f(t_{p^{-1}(p(1))}, \dots, t_{p^{-1}(p(n))}) \\ &= f(t_1, \dots, t_n), t_i \in T_i \end{aligned}$$

This clearly shows that f and f' compute the same for all inputs. Similar to the elimination of unused arguments no further work is required.

Renaming of Function Arguments

3.2 Ordering of the Canonicalization Steps

It is important to order the canonicalization-steps such that they do not interfere with previous once. In our case this is still relatively simple since we only have two constraints.

First, the argument renaming step should be run after the last modification to functions arguments. In our case this implies that it has to run after the elimination of unused arguments step and after the argument sorting step.

Second, the sorting of function arguments has to take place after the last modification to the function body in which the usage order of the arguments is changed. This means concretely that it has to be run after the step replacing greater than with less than operators.

Based on those two constraints we can define the following order:

1. Replacing Greater-Than with Less-Than
2. Non-strict Operator Replacement
3. Elimination of unused Function Arguments
4. Sorting of Function Arguments
5. Renaming of Function Arguments

3.3 Issues with the original version

4 Evaluation

4.1 Setup

For the evaluation we used an AMD Ryzen 9 5900 CPU which has 12 cores running at up to 4.8 GHz .

Each benchmarked program was build once using Minotaur with canonicalization disabled and once with canonicalization enabled. Since our canonicalization steps are compositional we measured the effect each step and combination of steps have. Furthermore, we ran the provided tests and benchmarked each compiled program.

We chose to benchmark libgmp-6.2.1 as the original paper did. Due to the long compile times experienced on libYUV, we decided against benchmarking it, like the original paper did, and instead opted to select different smaller projects.

4.2 Results

5 Discussion

6 Future Work

7 Conclusion

References

- [1] Zhengyang Liu, Stefan Mada, and John Regehr. “Mino-
taur: A SIMD-Oriented Synthesizing Superoptimizer”.
In: *Proceedings of the ACM on Programming Lan-
guages (OOPSLA2)* 8.OOPSLA2 (Oct. 2024). [https:
//doi.org/10.1145/3689766](https://doi.org/10.1145/3689766), pp. 1–25. DOI: 10.
1145/3689766.

Figure 3: Example applying each canonicalization step.

input:

```
define i1 @cut(i32 %a, float %b, i32 %c, float %d) {
    %1 = fcmp uge float %b, %d
    %2 = icmp sle i32 %c, 15
    %3 = and i1 %1, %2
    ret i1 %3
}
```

unused argument elimination:

```
define i1 @cut (float %b, i32 %c, float %d) {
    %1 = fcmp uge float %b, %d
    %2 = icmp sle i32 %c, 15
    %3 = and i1 %1, %2
    ret i1 %3
}
```

replacing greater-than with less-than:

```
define i1 @cut(float %b, i32 %c, float %d) {
    %1 = fcmp ule float %d, %b
    %2 = icmp sle i32 %c, 15
    %3 = and i1 %1, %2
    ret i1 %3
}
```

replacing non-strict operators:

```
define i1 @cut(float %b, i32 %c, float %d) {
    %1 = fcmp ule float %d, %b
    %2 = icmp slt i32 %c, 16
    %3 = and i1 %1, %2
    ret i1 %3
}
```

sorting arguments:

```
define i1 @cut (float %d, float %b, i32 %c) {
    %1 = fcmp ule float %d, %b
    %2 = icmp slt i32 %c, 16
    %3 = and i1 %1, %2
    ret i1 %3
}
```

renaming argument:

```
define i1 @cut (float %__canonArg, float %__canonArg1, i32 %__canonArg2) {
    %1 = fcmp ule float %__canonArg, %__canonArg1
    %2 = icmp slt i32 %__canonArg2, 16
    %3 = and i1 %1, %2
    ret i1 %3
}
```